Mathematics or the Verifier? No Detectable Gain from Verifier-Filtered Self-Training of a 1.5B Model

Tan Sang Nguyen[1] and Duy Anh Tran[1]

Ton Duc Thang University, Ho Chi Minh City, Vietnam
52300057@student.tdtu.edu.vn, 52300006@student.tdtu.edu.vn

Abstract. Self-training and reinforcement learning with verifiable rewards report large gains for small language models, but a rising score does not show that a model learned mathematics rather than what its verifier accepts. We study the cheapest form of the loop, generate–verify–select–train with offline LoRA SFT, on Qwen2.5-1.5B-Instruct with the rule-based verifier `math-verify`, on held-out MATH-500 and GSM8K exams (150 problems; 500 in a final round) at four learning rates on MATH-500 and two on GSM8K. No pass@1 or pass@k gain has a problem-level bootstrap interval excluding zero; in the final round, with a corrected recipe, pass@1 falls 4.8 points ($[-9.0, -0.6]$). The design detects only gains of about 6–10 points of pass@1 (80% power). A positive gate fails: training on reference solutions lowers pass@1 by 32.2 points, so the null concerns this recipe, not self-training in general. As a post hoc audit we track the share of the loop's own correct solutions that a storage defect in our pipeline rejects (about one in six). It falls by 1.5–2.6 points, which a falling `\boxed` share accounts for, so we find no adaptation to the defect but cannot exclude it. Two controls, patching a rarer defect and removing the filter, show no consistent difference.

Keywords: self-training · self-improvement · verifier bias · mathematical reasoning · small language models · verifiable rewards · negative result.

1 Introduction

A common recipe for improving the mathematical reasoning of a small language model (SLM) *without human annotation* is to let the model generate its own solutions, keep those an automated verifier judges correct, and fine-tune on them, for several rounds. We call this the *Generate–Verify–Train* (GVT) loop; it underlies STaR [20], ReST [13], ReST-EM [12] and rejection-sampling fine-tuning [18]. But the verifier never reads the reasoning: a rule-based verifier such as `math-verify`[1] only extracts a final answer and compares it with the gold, and

[1] https://github.com/huggingface/math-verify, version 0.9.0 with antlr4 4.13.2 in this work.

it rejects some correct answers written in forms it does not parse (Section 3.2). The training signal is "was the final string accepted", not "were the steps right".

The same gap runs through reinforcement learning with verifiable rewards (RLVR), which reports large gains at this scale: GRPO on a single training example lifts Qwen2.5-Math-1.5B from 36.0% to 73.6% on MATH-500 [15], a gain whose authors report only 8.6 points beyond format correction, and similar recipes improve DeepSeek-R1-Distill-Qwen-1.5B [3] and open base models from 0.5B up [21]. Yet Shao et al. [10] find that GRPO with *random* rewards raises Qwen2.5-Math-7B by 21.4 points on MATH-500, against 29.1 with ground-truth rewards. A rising score therefore does not by itself show that a model has learned to solve more problems, rather than to surface behaviour it already had or to produce what its reward accepts. In RLVR, per-step false-negative rates on the policy's own rollouts have been tracked [4,17], and Egashira et al. report that a policy first unlearns the behaviour that triggers false negatives. This paper studies the question in the cheapest and most widely used form of the loop, offline rejection-sampling SFT, where a false reject only removes a training example, and applies the same kind of per-round false-reject measurement there, post hoc, for one defect of our pipeline.

We ask: *after several GVT rounds, is the model better at mathematics, or has it only learned to write what the verifier accepts?* We fixed four measures in advance: (i) pass@1; (ii) pass@k; (iii) output style, chiefly the share of generations containing `\boxed`; and (iv) per-problem transitions (problems *added* and *lost* between rounds). The pre-set reading rule was that "learning the verifier's style" requires pass@1 to rise, pass@k to stay flat and style to converge at once. In hindsight the rule is one-sided: it can only fire when accuracy rises (Section 5.1). Only these measures and the rule were fixed in advance. The higher learning rates, the controls, the round count and k at those rates, sample accuracy as primary measure, the termination measures and the re-scoring for a second verifier defect, and the final 500-problem round (Section 4.6) were all added after seeing earlier runs; all are post hoc.

Our answer is negative on the first branch, within the power of the design: we detect no improvement at any learning rate, though gains below about 10 points of pass@1 or 3.5–5.5 of sample accuracy could have been missed, and no arm of this recipe improves at all. On the second branch the pre-set rule is silent. The post hoc audit of Section 5.3 finds no sign that the model learns to avoid one verifier defect, but it has little power to see such learning. What appears, most clearly at the highest rate, is lower accuracy on the solutions the model does finish and, clearly at only one of two seeds, more generations that run out of tokens. Two readings are not supported: a sharp decline in a pooled design does not reproduce under isolation (Section 4.1), and an isolated design at a learning rate of 10^{-5} changed the outputs little (Section 4.2).

Contributions.

- For a 1.5B model, LoRA SFT and one rule-based verifier, no gain is detectable on a held-out exam at any learning rate we tried, with problem-level

bootstrap intervals and the smallest effects the design could detect. No arm improves, not even training on reference solutions, so this is a null for the recipe, not for self-training.
– A post hoc per-round false-reject audit of one verifier defect, carried over from RLVR [4,17] to offline SFT self-training. The rejected share falls by 1.5–2.6 points, which a falling `\boxed` share accounts for, and is flat on the affected problems; the test has little power, and an injected-bias control shows no selective effect (Section 5.3).
– A decomposition of an accuracy decline under a fixed token cap into a finished-accuracy part and a cap-share part: at the highest rate, finished generations lose 5.5–10.6 points against the untrained model on the same problem mix, and the cap-hit share rises only at that rate.

2 Related Work

Self-training and its limits. STaR [20] introduced the generate–filter–retrain loop, and ReST [13], ReST-EM [12] and RFT [18] formalise it; ReST-EM, which like us restarts from the base model each iteration, reports diminishing gains after the first round. Yue et al. [19] show that RL raises pass@1 while the base model wins at large pass@k, and Teacher-Free Self-Training [14] concludes that self-training "amplifies but does not compound". Training repeatedly on model-generated data erodes distribution tails [11], and a narrow selector can accelerate collapse [9].

RL with verifiable rewards at small scale. RLVR with GRPO reports large gains near our model size: one training example lifts Qwen2.5-Math-1.5B on MATH-500 from 36.0% to 73.6% [15], about 7,000 examples lift DeepSeek-R1-Distill-Qwen-1.5B on AMC23 from 63% to 80% [3], and zero-RL works across ten open base models including Qwen2.5 from 0.5B [21]. These studies improve accuracy, which our SFT loop does not; of Wang et al.'s 37.6 points only 8.6 lie beyond format correction. Shao et al. [10] show that it need not be reward learning: random rewards raise Qwen2.5-Math-7B by 21.4 points on MATH-500, which they trace to amplified pre-existing behaviour, and the effect varies by model family. We do not compete with RLVR on accuracy; the audit of Section 5.3 asks of a loop whether the model has adapted to one verifier defect, and we apply it only to SFT.

Verifier error. TinyV [17] finds that rule-based verifiers reject many correct solutions; From Accuracy to Robustness [6] reports an average recall of 86%, and 0.95 for `math-verify` on their MATH subset. Imperfect Verifiers [1] models fixed false-accept and false-reject rates. Egashira et al. [4] find in RLVR that systematic false negatives behave like random noise whereas false positives can cause plateau or collapse. They also track the false-negative rate on the policy's rollouts per step and see the policy first unlearn the behaviour that triggers it; TinyV tracks the same share across GRPO steps. Our setting differs in paradigm

(offline SFT with a fresh adapter per round) and task, and our controls cannot test their findings (Section 4.3).

Auditing self-improvement. Xu et al. [16] audit LoRA self-training of Qwen3-8B against a frozen control and identify seven measurement failures, including training-seed variance and a fixed token cap under style drift, that each invert a finding when uncontrolled. This is the study closest to ours; it is why we separate finished accuracy from termination (Section 4.5). We differ in scale (a 1.5B model), in the verifier (a rule-based one whose rejection defect we measured) and in auditing what that verifier's filtering does to the training pool, not only whether accuracy moved. Kim et al. [8] show that self-distillation on ground-truth solutions can cut accuracy while shortening responses, as our train-on-gold control does. Unlike their distillation arm, our positive control fails (Section 4.6). We found no multi-round study combining a measured format-biased rule-based verifier with a patched-verifier control, but since our patch proved too weak (Section 4.3) we do not claim this combination as the contribution. The per-round false-reject measurement of Section 5.3 is not new either: it is the per-step tracking of Egashira et al. and TinyV, applied here to offline SFT self-training and to a defect of our own pipeline. The PRM800K grader [7] normalises `\dfrac` and `\frac43` and could serve as a second verifier; we did not run it. Supplementary Appendix **??** gives the full comparison.

3 Setup

3.1 The GVT Loop

Each round t has four steps. *Generate:* the current model samples k solutions per problem. *Verify:* `math-verify` extracts the final answer and compares it with the gold. *Select:* accepted solutions are kept. *Train:* a *fresh* LoRA adapter is attached to the *base* model and trained on the kept solutions for one epoch. Round 0 is the untrained base model; the round-t model ($t \geq 1$) is the base plus an adapter trained on the pool accepted at round $t-1$. Adapters are not accumulated, so earlier rounds affect later ones only through the data they generate; conclusions about (non-)compounding below apply to this restart-from-base design, which gives compounding less room than continued training would.

3.2 The Verifier and Its Directional Bias

A format probe that re-renders gold answers in individually valid forms shows that `\dfrac`/`\tfrac` are rejected 100% of the time on both datasets (the command is not parsed). On real solutions (one public Qwen2.5-7B-Instruct solution per problem for 200 MATH-500 problems, judged correct by a scripted normalised comparison) the false-negative rate is 7.8% (10 of 128; Wilson interval 4.3–13.8%) on MATH-500 and 0% on GSM8K. Eight of the ten fail when the whole solution is verified but pass when only the boxed answer is; most are compound answers, so they are probably largely the gold-storage defect described

below rather than extraction of a wrong token. The audit inspected only rejections, so it gives no false-positive rate. The verifier also cannot parse 51 of the 500 stored MATH-500 golds (10.2%); these problems always count as unsolved. That is a constant offset in counts within a partition, so it cancels in within-run and same-partition differences, but not in ratios or in levels compared across partitions (11–18 such problems per exam partition).

The patch. For the control we rewrote `\dfrac` and `\tfrac` to `\frac` before parsing, removing the 100%-rejection class and nothing else.

The gold-storage defect (post hoc). Our pipeline stores the gold without a delimiter, and `math-verify` applies its full LaTeX extraction only inside `\boxed{}` or `$...$`, so a correct boxed prediction can fail against a bare gold (e.g. the prediction `\boxed{\frac{4}{3}}` against the stored gold `\frac43`). Re-scoring the saved generations with a verifier V' that wraps the gold in `\boxed{}` when the prediction contains one shows that this defect rejects 15.5–18.3% of the generations V' accepts. It would enlarge the round-0 kept pool by about a fifth to a quarter (440 → 542, +23%), against 0–4 examples for the `\dfrac` patch, and it lowers absolute pass@1 by 4.7–8.0 points in self-training. It is bound to the problem, not to the answer: on the 54–62 problems where it ever fires, it rejects 94–98% of correct answers in every round (87–98% where V parses the gold), and 29–34 of these problems have a gold the original verifier cannot parse at all, so no answer form avoids it there. Re-scored, sample accuracy still falls in all five runs we re-scored; the original-verifier changes are 0.73–0.85 of the re-scored ones, an observed ratio that we do not use to rescale other numbers or intervals. All tables keep the original verifier, the one the loop trained with. Section 5.3 uses this defect for the adaptation audit; Supplementary Appendix **??** gives the full analysis.

3.3 Two Experimental Designs

Design P (pooled) loads both datasets (500 problems each) together and trains *one* adapter per round on the union of accepted solutions, with selection and evaluation on the same 1,000 problems. *Design I (isolated)* uses one dataset and one adapter per run, and holds out 30% of the 500 problems (150) as an `exam` set; generation runs on all 500 problems each round, but only the other 350 can enter the training pool, and all Design I pass rates are on the exam set. Table **??** summarises them.

Design I has three runs, all Qwen2.5-1.5B-Instruct, five rounds, $k = 8$, differing in one respect each: **Run A** (MATH-500, original verifier), **Run B** (MATH-500, patched verifier) and **Run C** (GSM8K, original verifier). Runs A and B were repeated at a second seed; since the seed also sets the train/exam partition, the repetition draws a different 150-problem exam set. Most of the later evidence comes from higher-rate runs introduced in Section 4.4; Table **??** (Supplementary Appendix **??**) lists every run. A "seed" sets the exam partition, the selection set and the training randomness at once; runs with the same seed share the partition. "Five rounds" means round 0 (the untrained model) and four adapters, each trained once on the previous round's pool; "three rounds" means round 0

and two adapters, so the round-2 model is a single adapter (about 195 optimizer steps for Run A at 5×10^{-4}) trained on the round-1 pool. All runs use one model family, Qwen2.5.

3.4 Models, Data, Training, Metrics

We use Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct (Design P uses both, Design I the 1.5B model only) and the first 500 problems of the GSM8K test split [2] and MATH-500, the 500-problem subset of MATH [5]. The prompt is a single user turn in the model's chat template: "Solve the math problem. Put the final answer in `\boxed{}`." followed by the problem. Generation samples at temperature 0.8 with a cap of 512 new tokens; the other decoding settings come from the checkpoint's generation configuration (top-p 0.8, top-k 20, repetition penalty 1.1). Training is LoRA SFT (`transformers` 5.0.0, `peft` 0.19.1, TRL 1.10.0 `SFTTrainer`; rank 16, $\alpha = 32$, on `q_proj` and `v_proj`, LoRA dropout and weight decay at the library defaults; batch size 2; one epoch; AdamW with linear learning-rate decay and no warmup; loss over the whole chat sequence, prompt included; fp16 on one Kaggle T4), with a training sequence limit of 512 tokens that includes the prompt. Because this limit equals the generation cap but also holds the prompt, any kept solution longer than about 512 tokens minus the prompt length is cut in training and loses its final answer and end-of-sequence token (24–26% of the round-0 pool; Section 4.5). All kept solutions are used, so the number of optimizer steps per round is half the pool size. The learning rate is 10^{-5} unless stated otherwise; this value was the default of our training code and was not tuned (Section 4.4). Metrics per round, on the exam set: pass@1 is the correctness of the *first* of the k samples (one draw per problem, so it carries sampling noise); pass@k; *sample accuracy*, the mean over all k samples; style counts over all generations (boxed, dfrac, dollar, ...); and per-problem added/lost counts between consecutive rounds, where a problem is *solved* in a round if at least one of its k samples is accepted. Added minus lost therefore equals $150 \times \Delta$pass@k, so these counts add only the gross churn behind the net change, not an independent test. pass@1 was fixed in advance; sample accuracy was not among the pre-set measures, but it estimates the same quantity from all k samples with less variance, so after the runs we chose it as the primary accuracy measure. Claims that rest on it, including the 2.2-point bound, are therefore post hoc in the choice of measure. "Correct" always means accepted by the verifier (preset `reward`), the same verifier that selects the training data.

Uncertainty. For every within-run change (round 0 against the last round) and every paired comparison of two runs on the same exam partition we report a 95% interval from a paired problem-level bootstrap (10,000 resamples of the 150 exam problems; 2,000 for the matched and audit columns), which respects the clustering of the k samples within a problem. These intervals do *not* cover training-seed variance, which we see only across two or three seeds: the one same-configuration replicate pair we have ends 3.2 points of sample accuracy apart. We make no multiplicity correction, so we rest conclusions on patterns across

runs and seeds, not on single intervals. Single proportions use Wilson intervals. *Detectable effects.* From the interval half-widths h in Table 2, a change would be detected with 80% power at the 5% level only if it exceeded about $2.8\,h/1.96$: about 10 points of pass@1, 8–10 of pass@k and 3.5–5.5 of sample accuracy on MATH-500. A null in this paper means no gain of roughly that size. Details are in Supplementary Appendix **??**.

4 Results

4.1 Design P: An Apparent Collapse

In the pooled design the 1.5B model declines severely and with acceleration (pass@1 50.0% $\rightarrow$ 18.6%, pass@8 72.6% $\rightarrow$ 48.6%; Table **??**, Supplementary Appendix **??**), on both datasets (Table **??**); the 0.5B decline is small. The accepted pool contracts every round and `\boxed{}` falls from 5,608 to 2,693 of 8,000 generations. This design cannot be read causally: one adapter trains on both datasets' pools; it scores on the problems it selects from; and it trains on about 4.5$\times$ as many examples per round as the isolated MATH-500 run (3,962 against 872 at round 0). No run separates these explanations (Supplementary Appendix **??**).

4.2 Design I at 10^{-5}: Small Declines, a Larger One on GSM8K

On the held-out exam (Fig. **??** and Table **??** in Supplementary Appendix **??**) pass@8 ends within 4 points of round 0 in every run. On MATH-500, pass@1 is 2.7–4.0 points lower at round 4, with every interval including zero (Table 2); sample accuracy falls 1.2–3.2 points, with an interval excluding zero only for B at seed 0 (A at seed 0 reaches 0.0), and matched finished accuracy 1.1–4.2, with intervals that only just exclude zero for A and B at seed 0, which are close to one replicate. We read this as a possible small decline, not an established one; what we do not see is a gain of the size the design could detect. Run C (GSM8K) declines more: pass@1 falls 12.7 points ($[-21.3, -4.0]$), almost all in finished accuracy, but it took about twice Run A's optimizer steps, so we do not attribute this to the dataset alone. On MATH-500 two coarse output statistics, length and cap-hit share, barely change, which is weak evidence that the adapter did little (Supplementary Appendix **??**).

4.3 The Patch Control Could Not Detect an Effect

Runs A and B differ only in the `\dfrac`/`\tfrac` patch. At round 4, B minus A is -0.7 [-3.3, $+2.0$] points of pass@1 at seed 0 and $+0.0$ [-2.7, $+2.7$] at seed 1 (recomputed from the saved generations; Table 2 lists values from the run logs that differ by one problem, giving -0.7). This null is uninformative: `\dfrac` appears in 0.50–0.73% of generations, so the kept pools differ by 3–4 examples (under 0.5% of the 872-example pool), and at 10^{-5} the adapter changes

the outputs little anyway. Removing one rare format defect had no detectable effect, which a prevalence count alone predicts; the frequent gold-storage defect is present in both arms.

4.4 Higher Learning Rates

Because 10^{-5} changed the MATH-500 outputs little, we reran Run A at 10^{-4} (three seeds), 2×10^{-4} (two) and 5×10^{-4} (two, with Run B), with three rounds and $k = 4$ (Table 1). These runs differ from the 10^{-5} runs in k, rounds, pool size and the pass@k metric as well as rate, so only 10^{-4} to 5×10^{-4} isolates the rate. *At* 5×10^{-4} *every run declines:* pass@1 falls 4.0–11.3 points and the sample-accuracy intervals exclude zero in all four A/B runs, though same-seed A and B are closer to two replicates than four. *The decline may grow with the rate on one seed:* at seed 1 sample accuracy falls 1.5, 3.5 and 8.3 points across the three rates, at seed 2 an almost flat 3.7, 3.8 and 4.7. The cap-hit share rises only at 5×10^{-4}. *The patch has no consistent effect:* B minus A in round-2 pass@1 is +5.3 at seed 1 and −7.3 [−14.0, −0.7] at seed 2, a sign flip from pools that differ by 0–3 examples: trajectory noise. *Style does not separate imitation from drift:* boxed share falls with the pool, which both would produce.

Table 1. Run A (MATH-500, original verifier, $k = 4$, three rounds) across learning rates and seeds: pass@1, pass@4 and sample accuracy (600 exam generations) at round 0 → round 2, the share of round-2 generations that hit the 512-token cap (decoded text re-tokenised to ≥ 508 tokens; round 0 is 43.0–43.2%, 48.2% at seed 3), and problems added/lost summed over rounds 1–2. The two 5×10^{-4} rows are the A rows of Table **??**.

Rate	Seed	pass@1	pass@4	Sample acc.	Cap-hit (R2)	Added/lost
10^{-4}	1	33.3 → 34.0	50.0 → 49.3	33.7 → 32.2	42.7%	20/21
10^{-4}	2	31.3 → 28.0	51.3 → 45.3	30.8 → 27.2	42.8%	25/34
10^{-4}	3	29.3 → 26.7	40.0 → 41.3	28.5 → 27.5	45.7%	16/14
2×10^{-4}	1	33.3 → 31.3	50.0 → 46.7	33.7 → 30.2	40.8%	23/28
2×10^{-4}	2	31.3 → 25.3	51.3 → 45.3	30.8 → 27.0	39.7%	21/30
5×10^{-4}	1	33.3 → 23.3	50.0 → 42.7	33.7 → 25.3	46.2%	22/33
5×10^{-4}	2	31.3 → 27.3	51.3 → 44.7	30.8 → 26.2	51.2%	26/36

Two controls at 5×10^{-4}. The *no-filter* run trains on all k samples of every selection problem (1,400 examples per round), removing the verifier; the *train-on-gold* run trains on the 350 reference solutions every round (Table 3). Neither is a single-variable control: no-filter takes 3–4× Run A's optimizer steps, train-on-gold about the same. Removing the filter is not better than keeping it beyond noise and is lower on pass@4 at both seeds (round-2 differences −2.0 and −5.3 points, intervals including zero), but the dose difference means this cannot be attributed to the filter; the no-filter pool is also truncated far more often in

Table 2. Change from round 0 to the last round (percentage points) with 95% paired problem-level bootstrap intervals over the 150 exam problems (10,000 resamples; `bootstrap_ci.py` in the code repository). Last round is R4 at 10^{-5} ($k = 8$) and R2 elsewhere ($k = 4$). "Finished, matched" is the last-round accuracy of generations below the 512-token cap minus the untrained model's accuracy on the same problem mix (Section 4.5; 2,000 resamples, `capanat.py` in the repository). The intervals do not include training-seed variance. Recomputed values match the reported ones except six round values that differ by one problem (Supplementary Appendix **??**), e.g. B, seed 1 at 10^{-5}, whose round-4 pass@8 is 58.0% against 58.7%; the matched column uses the separate re-scoring described there.

Run, seed	Δpass@1	Δpass@k	Δsample acc.	Finished, matched
Learning rate 10^{-5}				
A, 0	-2.7 $[-9.3, 4.0]$	-1.3 $[-8.0, 5.3]$	-2.4 $[-4.8, 0.0]$	-3.8 $[-7.2, -0.5]$
A, 1	-3.3 $[-10.0, 3.3]$	$+0.0$ $[-6.0, 6.0]$	-1.5 $[-4.1, 1.0]$	-2.3 $[-5.9, 1.2]$
B, 0	-3.3 $[-10.0, 3.3]$	-4.0 $[-10.7, 2.7]$	-3.2 $[-5.9, -0.6]$	-4.2 $[-7.8, -0.9]$
B, 1	-4.0 $[-10.7, 2.7]$	-1.3 $[-7.3, 4.7]$	-1.2 $[-3.7, 1.3]$	-1.1 $[-4.6, 2.6]$
C, 0	-12.7 $[-21.3, -4.0]$	-2.7 $[-7.3, 2.0]$	-13.1 $[-16.7, -9.8]$	-12.8 $[-16.5, -9.4]$
Learning rate 10^{-4}				
A, 1	$+0.7$ $[-6.7, 8.0]$	-0.7 $[-6.0, 4.7]$	-1.5 $[-5.3, 2.2]$	-4.3 $[-9.6, 1.1]$
A, 2	-3.3 $[-9.3, 2.7]$	-6.0 $[-12.7, 0.7]$	-3.7 $[-7.3, -0.2]$	-3.5 $[-9.2, 2.3]$
A, 3	-2.7 $[-10.0, 4.0]$	$+1.3$ $[-4.7, 7.3]$	-1.0 $[-4.3, 2.2]$	-0.6 $[-6.0, 4.7]$
Learning rate 2×10^{-4}				
A, 1	-2.0 $[-8.0, 4.0]$	-3.3 $[-10.0, 2.7]$	-3.5 $[-7.2, 0.0]$	-4.0 $[-8.9, 0.5]$
A, 2	-6.0 $[-12.7, 0.7]$	-6.0 $[-12.7, 0.7]$	-3.8 $[-7.5, -0.2]$	-4.7 $[-10.1, 1.0]$
Learning rate 5×10^{-4}				
A, 1	-10.0 $[-16.7, -3.3]$	-7.3 $[-14.0, 0.0]$	-8.3 $[-12.3, -4.5]$	-9.1 $[-14.2, -4.2]$
A, 2	-4.0 $[-11.3, 3.3]$	-6.7 $[-13.3, 0.0]$	-4.7 $[-8.2, -1.2]$	-5.5 $[-11.7, 0.1]$
B, 1	-4.7 $[-12.0, 2.7]$	-2.0 $[-8.7, 4.7]$	-6.7 $[-10.5, -2.8]$	-6.8 $[-12.4, -1.0]$
B, 2	-11.3 $[-17.3, -5.3]$	-11.3 $[-18.0, -4.7]$	-9.2 $[-12.8, -5.5]$	-5.9 $[-10.9, -1.0]$
No filter, 1	-6.0 $[-12.7, 1.3]$	-9.3 $[-16.7, -2.7]$	-8.2 $[-12.2, -4.3]$	-10.6 $[-16.4, -5.5]$
No filter, 2	-8.7 $[-16.0, -1.3]$	-12.0 $[-19.3, -4.7]$	-10.2 $[-14.2, -6.3]$	-6.6 $[-11.4, -2.0]$
Gold, 1	-18.0 $[-26.7, -10.0]$	-20.7 $[-28.7, -12.7]$	-19.7 $[-25.3, -14.2]$	-25.6 $[-32.1, -18.9]$
Gold, 2	-21.3 $[-28.7, -14.0]$	-20.7 $[-29.3, -12.7]$	-18.7 $[-23.7, -13.7]$	-25.9 $[-31.7, -20.0]$
C, 1	-22.0 $[-29.3, -15.3]$	-9.3 $[-15.3, -3.3]$	-15.3 $[-19.8, -11.2]$	-12.1 $[-16.6, -7.9]$

training (62.4% against 24.5% at round 0). Training on reference solutions is followed by a much larger decline (pass@4 down about 21 points at both seeds) while the model adopts the reference style (`$`-delimited spans 5% → 98%). Post hoc checks bound verifier error from below only (about 2 points of sample accuracy among boxed answers), and the 14% of round-2 generations without `\boxed` are not covered, so we think this fall is unlikely to be mainly verifier error but cannot bound it from above. That no arm, this one included, improves on the untrained model means we have no positive reference: the nulls above may reflect the recipe (LoRA on q/v projections, loss over the prompt, a 512-token limit) rather than self-training. Run C at 5×10^{-4} (GSM8K, one run) falls 22.0 points of pass@1, from both less accurate and more unfinished solutions (Supplementary Appendix **??**).

Table 3. Controls at 5×10^{-4} (MATH-500, $k = 4$, three rounds), on the exam partition of Run A at the same seed. Pool is the kept pool at round 0 → round 2; the round-2 model trains on the round-1 pool (392 and 386 examples, about 195 steps) and optimizer steps are half the pool size.

Run	Pool	pass@1			pass@4		
		R0	R1	R2	R0	R1	R2
Seed 1							
A (verifier-filtered)	440→326	33.3	32.7	23.3	50.0	49.3	42.7
No filter	1400	33.3	26.7	27.3	50.0	49.3	40.7
Train on gold	350	33.3	18.0	15.3	50.0	33.3	29.3
Seed 2							
A (verifier-filtered)	424→350	31.3	31.3	27.3	51.3	46.0	44.7
No filter	1400	31.3	34.7	22.7	51.3	48.7	39.3
Train on gold	350	31.3	8.7	10.0	51.3	28.7	30.7

4.5 Anatomy of the Decline

This subsection is post hoc and descriptive. Sample accuracy mixes whether a finished solution is right with whether the model finishes within the 512-token cap at all. Re-tokenising the saved text, we call a generation *cap-hit* at ≥ 508 tokens. At the rates where the share does not rise, 40–46% of round-2 MATH-500 generations already hit the cap (Table 1), so the cap binds from the start. At 5×10^{-4}, compared with the untrained model on the same problem mix, finished generations lose 5.5–10.6 points, with intervals excluding zero in five of six runs (the six include two same-seed A/B pairs, which are close to replicates), while the cap-hit share rises 2–14 points. The matched measure conditions on which problems finish after training, so it is descriptive. A three-part decomposition attributes 12–30% of the decline to the cap share in the three original seed-1 runs (and −2 and 5% in two further runs) and 52–72% at seed 2 (Table **??**, Supplementary Appendix **??**). At 10^{-4} and 2×10^{-4} the cap-hit share does not rise and finished accuracy carries the whole change. Train-on-gold is different: responses shorten by more than half and finished accuracy collapses. Retraining without truncated examples, or re-scoring at a 1,536-token cap, leaves the deficit in place (Supplementary Appendix **??**).

4.6 A Final Round with a Corrected Recipe

Because no earlier arm improved, we ran one last round with the recipe changes that remove the suspected causes: loss on the completion only, LoRA on all linear layers, a 1,024-token cap and training limit, normalised gold answers, and a pool of 300 problems from the MATH training split, so that the exam is all 500 MATH-500 problems. It is one seed at 10^{-4} with $k = 2$ and two rounds. The verifier-filtered loop (298 examples kept) is compared with training on the 300 reference solutions, a positive gate that must raise accuracy for the recipe

to count as able to help. Intervals are paired problem-level bootstrap intervals over the 500 problems (10,000 resamples).

Table 4. Final round, MATH-500 (500 exam problems, $k = 2$, 10^{-4}, one seed). Change from the untrained model to round 1 in points, with 95% paired problem-level bootstrap intervals that omit training-seed variance.

Arm	pass@1	pass@2	Sample acc.
Untrained	49.2	59.4	–
Verifier-filtered loop	-4.8 $[-9.0, -0.6]$	-3.0 $[-6.8, +0.6]$	-2.9 $[-5.8, 0.0]$
Train on gold	-32.2 $[-36.8, -27.6]$	-32.0 $[-36.6, -27.2]$	-30.2 $[-33.8, -26.6]$

The gate fails. Training on reference solutions lowers pass@1 from 49.2% to 17.0%, and the model adopts the reference style ($-delimited spans in 60 of 1,600 generations at round 0, 1,577 at round 1, counted over the exam and selection problems), as in the earlier train-on-gold arms. A rerun at 2×10^{-5} fails the same way (49.2% $\rightarrow$ 20.6%, $-spans in 1,558 of 1,600 generations), so the failure is the reference style, not the rate, and the corrected recipe still has no positive reference. The verifier-filtered loop loses 4.8 points of pass@1, with an interval that only just excludes zero and a pass@2 interval that includes it; with one seed we read it as no gain. Half-widths of about 4 points of pass@1 (and 3.8 of pass@2) give detectable effects of about 6 and 5 points ($2.8h/1.96$), against about 10 before.

Injected bias. As a positive control for the audit we reran the loop once (seed 1, 10^{-4}) with a selection verifier that rejects every `\boxed` answer on a hash-chosen 30% of problems (exam verifier clean). The `\boxed` share on exam generations falls 4.9 points on the affected problems and 3.1 on the others, against $+0.1$ to -1.6 in the unbiased run; the group difference relative to the unbiased run is -0.1 ($[-6.1, 6.0]$, bootstrap). The model drops `\boxed` in general, not selectively; one seed cannot show avoidance either way.

5 Discussion

5.1 Answer to the Research Question

Does the model get better at mathematics? Not detectably. No run raises pass@1 or pass@k with an interval excluding zero, at any rate; in the 500-problem final round pass@1 falls 4.8 points. The sample-accuracy bound of about 2.2 points is read off observed intervals, uses a measure chosen after the runs and omits seed variance; on the pre-set measures the design could detect only about 10 points of pass@1 and 8–10 of pass@k (6 and 5 in the final round). Because no arm improves and the positive gate failed, the null may belong to this recipe, not to self-training.

Does it learn to write what the verifier accepts? The data cannot settle this. The pre-set pattern requires pass@1 to rise, which never happens detectably (the

largest rise is +0.7 [−6.7, +8.0]), so the rule's silence is not evidence against verifier-style learning. The post hoc audit of Section 5.3 finds no sign of adaptation to the gold-storage defect, but it is too weak to rule adaptation out.

What happened to the pooled collapse? It does not reproduce when each dataset has its own adapter and is scored on held-out problems; since the pooled design also trained on about 4.5× more examples per round, we cannot say whether pooling, selection-set scoring or dose produced it.

5.2 Interpretation

One explanation of the Design P curves is a self-reinforcing shrinking pool shaped by the biased verifier. The data neither support nor rule it out: the patch tests a rare bias, not a frequent one, and the no-filter control declines without any verifier but with more dose. The defensible statement is narrow: *removing a rare format defect, or the filter entirely, made no consistent difference at this resolution.* The frequent gold-storage defect was not varied in the main runs, and the no-filter control admits unstructured wrong answers, not the systematic false positives Egashira et al. find harmful, so neither tests their findings for RLVR.

5.3 A Proposed Audit for GVT with a Biased Verifier

Our pre-set rule tied "learning the verifier's style" to rising accuracy, so it could not speak once accuracy fell. From post hoc analyses we propose four checks for self-training with a rule-based verifier, as a protocol for future studies, not as tested methods.

(i) A per-round false-reject audit. Fix a repaired verifier V' that corrects a known defect class of the training verifier V. At each round t, on the loop's own generations, compute

$$r_t = \frac{\#\{\text{accepted by } V' \text{ but rejected by } V\}}{\#\{\text{accepted by } V'\}}.$$

If the model learns to write what V accepts, r_t should fall over rounds, whatever happens to accuracy. This is the per-step tracking of RLVR studies [4,17] applied to offline SFT; overall r_t is not enough on its own (Table 5; `rt_audit.py` in the repository). For the gold-storage defect it falls in all three self-training runs we re-scored, by 2.6, 2.6 and 1.5 points, with intervals reaching or just touching zero. But V' differs from V only on boxed predictions, so unboxed correct answers enter only the denominator, and the fall matches the falling `\boxed` share among correct answers (91 → 76% and 93 → 78% at 5×10^{-4}). On boxed answers alone r_t is flat (changes −0.4 to +0.7 points). The defect is also bound to the problem: it fires on 54–62 problems, rejecting 94–98% of correct answers (87–98% where V parses the gold), and 29–34 of them have an unparseable gold, so no answer form escapes it. On the 24–28 affected problems where some form could, r_t moves by −5.5, −1.6 and −0.2 points, with intervals 10–26 points wide.

A model adapting to the defect would have to write these answers unboxed, and the training pool holds only 2–12 examples per round from these problems that do so. Under SFT the pressure to adapt is therefore weak. Our one injected-bias control (Section 4.6) shows no selective avoidance either, so we cannot tell whether the audit would detect adaptation. We conclude only that we see none, in one defect class, three runs and three to five rounds. Train-on-gold, with no verifier in the loop, gives r_t rises of 2.1–3.3 points as its outputs become more boxed, so r_t moves with format for reasons unrelated to the verifier. It should be reported on the answers and problems where V and V' can differ and an accepted form exists, with the count of training examples that could teach avoidance.

Table 5. The false-reject audit for the gold-storage defect (post hoc), in percent (all 500 problems), round 0 → last round, with the change and a 95% problem-level bootstrap interval (2,000 resamples). "Boxed only" restricts to predictions containing `\boxed`; "affected, parseable" to problems where the defect ever fires and V can parse the gold. Pool: examples per round from affected problems that V accepts (and so avoid the defect).

Run	r_t subset	R0	Last	Δ	95% interval	Pool
A, 5×10^{-4}, seed 1	all	18.3	15.7	−2.6	$[-5.3, 0.3]$	3–5
	boxed only	20.0	20.7	+0.7	$[-2.9, 4.3]$	
	affected, parseable	92.8	87.2	−5.5	$[-20.1, 6.3]$	
A, 5×10^{-4}, seed 2	all	18.1	15.5	−2.6	$[-5.2, 0.1]$	2–3
	boxed only	19.4	19.8	+0.4	$[-2.8, 3.4]$	
	affected, parseable	95.1	93.5	−1.6	$[-14.7, 9.5]$	
A, 10^{-5}, seed 0	all	18.3	16.8	−1.5	$[-2.9, 0.0]$	9–12
	boxed only	19.9	19.5	−0.4	$[-2.0, 1.2]$	
	affected, parseable	92.3	92.0	−0.2	$[-4.9, 4.6]$	
Gold, seed 1	all	18.2	21.5	+3.3	$[-1.2, 8.1]$	—
	boxed only	20.0	22.8	+2.8	$[-2.0, 7.7]$	
	affected, parseable	95.5	94.7	−0.7	$[-9.1, 6.2]$	
Gold, seed 2	all	18.0	20.2	+2.1	$[-2.2, 6.7]$	—
	boxed only	19.4	21.1	+1.7	$[-3.3, 6.5]$	
	affected, parseable	95.1	97.6	+2.5	$[-4.8, 11.9]$	

(ii) Decompose declines under a fixed cap. Report the change in sample accuracy split into a finished-accuracy part and a cap-share part (Table **??**, Supplementary Appendix **??**). Here the split is descriptive and its balance differs by partition: at 5×10^{-4} the cap share accounts for 12–30% of the decline in the three original seed-1 runs and 52–72% at seed 2.

(iii) Check the dose of a control before running it. Count how many training examples the treatment changes. Our `\dfrac` patch changed 0–4 examples, while the unvaried gold-storage defect would have changed about a fifth to a quarter

of the pool (440 → 542 at round 0); the first number alone shows the patch control could not detect an effect.

(iv) Match training dose across designs. Our pooled design trained on about 4.5× more examples per round than the isolated one, so its sharp decline cannot be attributed to pooling.

Check (i) in RLVR. There a false reject gives the correct answer zero reward, a direct pressure that SFT lacks, so the audit could be run on public RLVR checkpoints with the conditioning above; we have not done so.

6 Limitations

Unequal training doses confound every cross-arm comparison, and the controls are underpowered: the patch changes 0–4 training examples, and the no-filter and train-on-gold controls have two seeds each. *Intervals cover only problem draw and sampling,* not training-seed variance; several conditions have one seed, and no multiplicity correction is applied. *The learning-rate grid is coarse* and changes k, rounds, dose and metric along with the rate. *The power is limited:* with 150 exam problems only gains of about 10 points of pass@1 would be detected (about 6 with the 500 problems of Section 4.6, one seed, $k = 2$, two rounds). *The adaptation audit is weak:* 24–28 problems, 2–12 training examples per round that could teach the avoidance, and its one positive control (injected bias, one seed) showed no selective effect. *The verifier audit* used other models' solutions and inspected only rejections; the gold-storage defect was measured afterwards and ran unvaried through every run before the final round, and every headline number is graded by the biased verifier that also selects the data. *Truncation:* the 512-token cap binds from the start and truncates a quarter of the pool in the main runs. *Scope:* one model family and size, all instruction-tuned (no base model), one verifier, a restart-from-base design that leaves compounding little room (no continued training), and possible test-set contamination. *No arm improves on round 0, and the gate on reference solutions failed even with the corrected recipe,* so we have no positive reference showing that any recipe can help a 1.5B model here. The Supplement lists the rest.

7 Conclusion

We asked whether generate–verify–select–train iterations make a 1.5B model better at mathematics or only teach it to write what a rule-based verifier accepts. On held-out exams the first answer is no gain that this design could detect: no run shows a pass@1 or pass@k gain whose interval excludes zero, though gains below about 10 and 8–10 points could have been missed (6 in a final 500-problem round, where pass@1 falls 4.8 points), and the sample-accuracy bound of about 2.2 points is loose because it omits training-seed variance. No arm improves, and training on reference solutions lowers pass@1 by about 30 points even with a corrected recipe, so this is a null for the recipe, not for self-training. The second question stays open: our pre-set rule could only fire if accuracy rose, and the post

hoc `\boxed` share falls in a way that imitation of the less-boxed pool and drift would both produce. A post hoc per-round false-reject audit, carried over from RLVR, finds no sign that the model learns to avoid one verifier defect once the falling `\boxed` share is accounted for, but with about 25 affected problems, a handful of training examples per round and an injected-bias control that showed no selective effect, it cannot rule it out. At the highest rate the model is less accurate on the solutions it finishes, and more generations hit the cap at one of two seeds. Patching a rare verifier defect made no difference of consistent sign across seeds, removing the filter gave no gain, neither control could have detected a small effect, and a frequent defect in our gold storage, which rejects about one in six correct solutions, ran unvaried through every run before the final round. The pooled design's sharp decline does not reproduce under isolation; it also trained several times longer per round, so its cause is open.

A rising score, in self-training or in RLVR, does not by itself show that a model learned mathematics rather than its verifier. A per-round false-reject audit asks part of that question whether accuracy rises or falls, but it must be conditioned on answer format and on the problems where the defect can be avoided, and it needs a positive control; applying it to public RLVR checkpoints, where the pressure to adapt is direct, is a natural next step. A dose-matched control for the gold-storage defect and a recipe that passes the positive gate come first (Supplementary Appendix **??**).

Acknowledgments. This work was carried out as an undergraduate research project. No external funding was received.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Code availability Code, saved generations and analysis scripts are at https://github.com/tsan2711/DACNTT_2026 (tag `paper-2026-11`). Generation-time token counts, Design P's training logs and the 1,536-token re-scoring outputs were not saved.

Use of AI tools The authors used Claude (Anthropic), through Claude Code, to help write the manuscript, the analysis and training scripts, and to run automated reviews. The authors designed the study, ran the experiments and take responsibility for the content.

References

1. Cai, X.-Q., Wang, W., Liu, F., Liu, T., Niu, G., Sugiyama, M.: Reinforcement Learning with Verifiable yet Noisy Rewards under Imperfect Verifiers. arXiv:2510.00915 (2025)
2. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J.: Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 (2021)
3. Dang, Q.-A., Ngo, C.: Reinforcement Learning for Reasoning in Small LLMs: What Works and What Doesn't. arXiv:2503.16219 (2025)

4. Egashira, K., Vero, M., Dekoninck, J., Dörner, F.E., Staab, R., Vechev, M.: Delay, Plateau, or Collapse: Evaluating the Impact of Systematic Verification Error on RLVR. arXiv:2605.02909 (2026)
5. Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J.: Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874 (2021)
6. Huang, Y., Zeng, W., Zeng, X., Zhu, Q., He, J.: From Accuracy to Robustness: A Study of Rule- and Model-based Verifiers in Mathematical Reasoning. arXiv:2505.22203 (2025)
7. Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., Cobbe, K.: Let's Verify Step by Step. arXiv:2305.20050 (2023)
8. Kim, J., Luo, X., Kim, M., Lee, S., Kim, D., Jeon, J., Li, D., Yang, Y.: Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs? arXiv:2603.24472 (2026)
9. Qiao, X., Du, X., Liu, W., Zhang, J., Mai, P., Zhang, M., Pang, Y.: When Sample Selection Bias Precipitates Model Collapse. arXiv:2606.13732 (2026)
10. Shao, R., Li, S.S., Xin, R., Geng, S., Wang, Y., Oh, S., Du, S.S., Lambert, N., Min, S., Krishna, R., Tsvetkov, Y., Hajishirzi, H., Koh, P.W., Zettlemoyer, L.: Spurious Rewards: Rethinking Training Signals in RLVR. arXiv:2506.10947 (2025)
11. Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., Gal, Y.: AI models collapse when trained on recursively generated data. Nature (2024). https://doi.org/10.1038/s41586-024-07566-y
12. Singh, A., Co-Reyes, J.D., Agarwal, R., et al.: Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models. arXiv:2312.06585 (2023)
13. Gulcehre, C., Le Paine, T., Srinivasan, S., et al.: Reinforced Self-Training (ReST) for Language Modeling. arXiv:2308.08998 (2023)
14. Strozzi, I.L.: Teacher-Free Self-Training Amplifies but Does Not Compound: A Pass@k Crossover on a Free-Verifier Domain. arXiv:2606.07856 (2026)
15. Wang, Y., Yang, Q., Zeng, Z., Ren, L., Liu, L., Peng, B., Cheng, H., He, X., Wang, K., Gao, J., Chen, W., Wang, S., Du, S.S., Shen, Y.: Reinforcement Learning for Reasoning in Large Language Models with One Training Example. In: NeurIPS (2025). arXiv:2504.20571
16. Xu, C., Yan, N., Chen, L., Kechadi, M.-T.: Phantom Gains: Auditing Self-Improvement Against a Measured Null. arXiv:2608.20290 (2026)
17. Xu, Z., Li, Y., Jiang, F., Ramasubramanian, B., Niu, L., Lin, B., Poovendran, R.: TinyV: Reducing False Negatives in Verification Improves RL for LLM Reasoning. arXiv:2505.14625 (2025)
18. Yuan, Z., Yuan, H., Li, C., Dong, G., Lu, K., Tan, C., Zhou, C., Zhou, J.: Scaling Relationship on Learning Mathematical Reasoning with Large Language Models. arXiv:2308.01825 (2023)
19. Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Yue, Y., Song, S., Huang, G.: Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv:2504.13837 (2025)
20. Zelikman, E., Wu, Y., Mu, J., Goodman, N.D.: STaR: Bootstrapping Reasoning With Reasoning. arXiv:2203.14465 (2022)
21. Zeng, W., Huang, Y., Liu, Q., Liu, W., He, K., Ma, Z., He, J.: SimpleRL-Zoo: Investigating and Taming Zero Reinforcement Learning for Open Base Models in the Wild. arXiv:2503.18892 (2025)